RISC-V based Vectorization of Classic McEliece Key Generation

Mahnaz Namazi Rizi[1], Nuša Zidarič[1], Lejla Batina[2], and Nele Mentens[1,3]

[1] Leiden University, Leiden, Netherlands
{m.namazi.rizi,n.zidaric,n.mentens}@liacs.leidenuniv.nl
[2] Radboud Univerity, Nijmegen, Netherlands
lejla@cs.ru.nl
[3] KU Leuven, Leuven, Belgium
nele.mentens@kuleuven.be

Abstract. Quantum computers can break or weaken classical cryptography using Shor's and Grover's algorithms. This threat drives the development of post-quantum cryptography (PQC) algorithms, such as the Classic McEliece (CM) algorithm, which resists quantum attacks by relying on hard problems in coding theory. However, the complex computation and large key size make it challenging to implement CM in an efficient way in terms of computational performance. This research is the first to thoroughly explore and implement RISC-V Vector Extensions (RVV) for the acceleration of the CM key generation process. First, an evaluation is done of auto-vectorized and manually vectorized implementations of CM using the RISC-V Vector Extension Version 1.0 (RVV1.0), based on multiple vector register configurations. Further, several new custom RVV instructions are proposed to speed up the implementation even more. The presented work gives insight into the practical implementation capabilities of RVV for the speed-up of CM key generation on an FPGA.

Keywords: Classic McEliece, RVV, RISC-V, Custom vector extensions

1 Introduction

Quantum computers pose a significant threat to classical cryptographic algorithms due to their ability to perform certain calculations much faster than traditional computers. The security of traditional public-key algorithms, like RSA [26] and DSA [6] relies on the computational hardness of, e.g., integer factorization and discrete logarithms. These problems are practically infeasible to solve efficiently with classical computing. However, quantum algorithms like Shor's algorithm [29] can solve these problems in polynomial time, effectively breaking the security of traditional public-key algorithms. This has prompted a global effort to develop quantum-resistant cryptographic algorithms to ensure data security in a quantum era. In 2017, the National Institute of Standards and Technology (NIST) initiated a competition to evaluate and standardize quantum-resistant

key encapsulation mechanisms, public-key encryption algorithms, and digital signature schemes [22]. At the end of the third round of the competition, four algorithms were chosen for standardization, and four other algorithms were advanced to the fourth round for further study. One of the candidates in the fourth round is Classic McEliece (CM) [8]. CM is a code-based cryptosystem with relatively short ciphertext and large public key sizes in comparison to other NIST candidates [12, 5]. CM consists of three algorithms: key generation, key encapsulation, and key decapsulation. According to [8], key encapsulation and decapsulation are reasonably fast in software and impressively fast in hardware. The authors in [8] also mention that key generation is quite fast in hardware but relatively slow in software. When frequent key generation is required, like in TLS, key generation acceleration becomes critical [28]. Based on the implementations submitted to the second round of the NIST competition, each key generation attempt takes 800× to 2500× longer than encapsulation and 200× to 1200× longer than decapsulation for the different parameter sets [8]. Therefore, our work concentrates on accelerating CM key generation. Our approach relies on the use of standard and custom RISC-V Vector Extensions (RVV) [4]. The RISC-V instruction set architecture (ISA) is chosen for its open-source, royalty-free characteristics [35] and its ability to enhance performance through the addition of custom instructions. Moreover, RVV is vector length agnostic, which makes it highly flexible. To the best of our knowledge, the acceleration of CM key generation on RISC-V platforms via vectorization has not been thoroughly investigated or assessed on an actual implementation platform, such as an FPGA. We validate vector-accelerated designs on an open-source Ibex processor [17] in combination with the Vicuna vector coprocessor, which was proposed by Platzer and Puschner in [25] to speed up data-parallel applications. We compare the cycle counts, instruction counts, and memory footprints of the `ref` code, proposed by the authors of CM in [8], running on Ibex only with the vectorized implementations running on the Ibex-Vicuna processor-coprocessor architecture. We propose and evaluate three versions: (1) a scalar implementation based on the `ref` code from [8], compiled for RISC-V integer instructions; (2) a standard vector accelerated implementation, which we call `sva`, based on standard RVV1.0 instructions; (3) a custom vector accelerated implementation, which we call `cva`, based on custom vector instructions that are added to the `sva` code. We validate the three implementations with Spike [2], the official RISC-V ISA simulator. We evaluate and compare the performance and resource utilization of each implementation on an FPGA. As such, this work serves as a starting point for research that intends to optimize RISC-V based custom vector architectures. In Section 2, an introduction to RVV and CM key generation is given. In Section 3, we provide a review of previous implementations of CM key generation targeting high performance and/or low memory or resource utilization. The details of our approach and implementation are described in Section 4. Section 5 provides the results and comparison of all implementations, and finally, we conclude this paper in Section 6.

2 Background

2.1 RISC-V Vector Extensions

RISC-V was originally designed at the University of California, Berkeley, to provide an open-source reduced instruction set to support computer architecture research [35]. The RISC-V ISA supports 32/64/128-bit designs and is royalty-free. These features make RISC-V attractive for academia and industry. Although the RISC-V basic integer instruction set (I) consists of only 50 instructions, it can be extended by standard or non-standard instruction set extensions (ISE).

RISC-V Vector Extension Version 1.0 (RVV1.0) [4] was ratified in 2021. The RVV instructions can be categorized into five main groups: configuration setting instructions, loading/storing instructions, arithmetic instructions, permutation/reduction instructions, and mask instructions. RVV1.0 also comes with 32 additional vector registers, V_0-V_{31}, and seven additional unprivileged control and status registers (CSR). Three parameters determine the configuration of the vector registers. The first parameter, VLEN, defines the width of a single vector register in bits; it is a power of two no greater than 2^{16}, which must be fixed in the hardware description of the architecture and given as a flag to the compiler. The second parameter, LMUL, is the vector length multiplier, which defines the number of consecutive vector registers that are grouped to form a single vector on which vector instructions can be applied. LMUL can take integer values of 1, 2, 4, and 8 and fractional values of 1/2, 1/4, and 1/8. The third parameter, ELEN, defines the maximum size of each element of a vector register in bits, which must be a power of two, ELEN>=8, and VLEN>=ELEN [4]. LMUL and ELEN are defined in the software code using configuration-setting instructions.

2.2 Classic McEliece Key Generation

Classic McEliece, as a Key Encapsulation Mechanism (KEM), is used to establish a 32-byte session key for secure communication between a client and a server. KEMs consist of three algorithms: key generation, key encapsulation, and key decapsulation. To establish a session key, the server initially generates a pair of keys, a private key (SK) and a public key (PK), by employing the key generation algorithm. Then, the client generates a session key and encapsulates it as ciphertext using the server's PK. Finally, the server receives the client's ciphertext and employs its SK with the decapsulation algorithm to retrieve the session key.

CM is built on error correction codes and, for the public key, uses a parity check matrix (PCM) with binary Goppa codes over a finite field $\mathbb{F}_{2^m}$. A finite field, also known as a Galois field (denoted as $\mathbb{F}_q$), is a set of q elements equipped with two operations. Here, q is the field size and the two operations (typically addition and multiplication) satisfy certain properties. If q equals 2, $\mathbb{F}_2$ is called a binary field with two elements, zero and one. This field can be extended to $\mathbb{F}_{2^m}$ by using an irreducible polynomial of degree m over $\mathbb{F}_2$, and similarly, $\mathbb{F}_{2^m}$ can be extended to $\mathbb{F}_{2^{mt}}$ by means of an irreducible polynomial of degree t over $\mathbb{F}_{2^m}$. Let $g(x) = g_t x^t + ... + g_1 x + g_0 \in \mathbb{F}_{2^m}[x]$ and $L = \{\alpha_1, \alpha_2, ..., \alpha_n\} \in \mathbb{F}_{2^m}$

with $n \leq 2^m$ such that $g(\alpha_i) \neq 0$ for all $\alpha_i \in L$, then the Goppa code, as a linear error correction code, is denoted by $\Gamma = (g, L)$, where $g(x)$ is the Goppa polynomial and L is the support list. This Goppa code consists of all codewords $c = (c_1, c_2, ..., c_n) \in \mathbb{F}_2^n$ where $s(x) = \sum_{i=1}^{n} \frac{c_i}{x-\alpha_i} \equiv 0 \mod g(x)$. In decoding Goppa codes, the error-locating polynomial is derived from $s(x)$, where its roots indicate the error positions.

In CM, a number of parameters specify the security strength and the implementation properties of the algorithm:

- A positive integer m, which defines the field size as $q = 2^m$.
- A positive integer n, with $n \leq 2^m$, which is the codeword size.
- An integer $t \geq 2$, with $n > mt$, that specifies the error correction capability.
- A monic irreducible polynomial $f(x) \in \mathbb{F}_2[x]$ of degree m, which defines the representation $\mathbb{F}_2[x]/(f(x))$ of the field $\mathbb{F}_{2^m}$.
- A monic irreducible polynomial $F(y) \in \mathbb{F}_q[y]$ of degree t, which defines the representation $\mathbb{F}_q[y]/(F(y))$ of the field $\mathbb{F}_{2^{mt}}$.

Five parameter sets are proposed in systematic form, complemented by five others in semi-systematic form. Our work focuses on the `mceliece348864` parameter set in systematic form, with m=12, n=3488 and t=64, and with the following irreducible polynomials: $f(x) = x^{12} + x^3 + 1$ and $F(y) = y^{64} + y^3 + y + x$.

The CM key generation algorithm consists of the following steps (based on [8]):

Step 1 - RandomBytes: In this step, a 256-bit random value δ, which is called the seed, is generated by the RandomBytes algorithm. The `ref` code from the authors of CM [8] suggests using AES [21] for RandomBytes.
Step 2 - PRNG: The seed δ is fed to a Pseudorandom Number Generator (PRNG) to generate $E = PRNG(\delta)$, which consists of $(n + \sigma_2 * 2^m + \sigma_1 * t + l)$ random bits. In the `ref` implementation, the SHAKE256 [3] algorithm is used as the PRNG. The values $l = 256$, $\sigma_1 = 16$ and $\sigma_2 = 32$ are fixed for all CM parameter sets [8]. After generating E, we put δ' equal to the last l bits of E and define s as the first n bits of E.
Step 3 - FieldOrdering: In this step, a random permutation of all elements in $\mathbb{F}_{2^m}$ is generated. Later, n consecutive elements of this list will be used as the support list to generate the parity check matrix. It starts from bit $(n + 1)$ of E, takes each consecutive 32 bits as an integer, and repeats this until it has a sequence of $q = 2^m$ integers: $\{a_0, a_1, ..., a_{q-1}\}$. These integers must be distinct; otherwise, the whole key generation process is restarted from Step 2 after overwriting δ with the new seed, δ', generated at the end of that step. In order for the decoding algorithm (as part of the decapsulation) to be more efficient in terms of error-localization and error-value computation, the support list needs to be sorted. If we assign each element a_i in $\mathbb{F}_{2^m}$ a binary vector of length m and order them as integers in binary representation, we get the list $\{\alpha_1, ..., \alpha_q\}$ where $\alpha_{i+1} = \sum_{j=0}^{m-1} \pi(i)_j . z^{m-1-j}$ for all $i \in \{0, 1, \ldots, q-1\}$. Here, $\pi(i)$ is defined as a permutation of $\{0, 1, \ldots, q-1\}$ and $\pi(i)_j$ denotes the jth least significant bit of $\pi(i)$.

Step 4 - Irreducible: The Irreducible algorithm is used to generate the Goppa polynomial, $g(x)$, as a monic irreducible polynomial of degree t over $\mathbb{F}_{2^m}[x]$. This algorithm puts $f \in \mathbb{F}_{2^{mt}}[y]$ equal to the first mt bits of the $16t$ bits of E. The polynomial f is represented as $f = f_{t-1}y^{t-1} + \cdots + f_1 y + f_0$, where $f_i \in \mathbb{F}_{2^m}$ for $0 \leq i \leq t-1$. The matrix M of size $t \times t$ over $\mathbb{F}_{2^m}$ is formed in Equation 1, through repeated multiplications in $\mathbb{F}_{2^{mt}}$.

$$M = \begin{pmatrix} f_0 & f_1 & \cdots & f_{t-1} \\ f_0^2 & f_1^2 & \cdots & f_{t-1}^2 \\ \vdots & \vdots & \ddots & \vdots \\ f_0^t & f_1^t & \cdots & f_{t-1}^t \end{pmatrix} \quad (1) \qquad H = \begin{pmatrix} \frac{1}{g(\alpha_1)} & \frac{1}{g(\alpha_2)} & \cdots & \frac{1}{g(\alpha_n)} \\ \frac{\alpha_1}{g(\alpha_1)} & \frac{\alpha_2}{g(\alpha_2)} & \cdots & \frac{\alpha_n}{g(\alpha_n)} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\alpha_1^{t-1}}{g(\alpha_1)} & \frac{\alpha_2^{t-1}}{g(\alpha_2)} & \cdots & \frac{\alpha_n^{t-1}}{g(\alpha_n)} \end{pmatrix} \quad (2)$$

After the generation of matrix M, we need to apply Gaussian Elimination (GE) to this matrix and return the last row of the reduced matrix as the coefficients of Goppa polynomial g, if g has degree t; otherwise, the whole key generation process needs to start over from Step 2 after overwriting δ with the new seed, δ', generated at the end of that step. Established formulas for counting the number of irreducible polynomials indicate that the Irreducible algorithm has a success rate exceeding 99% when $t \geq 16$ [8].

Step 5 - MatGen: The binary Goppa code, defined in Step 3 and Step 4 enters the MatGen algorithm to generate the public key, T. Firstly, a $t \times n$ matrix $H = \{h_{ij}\}$ over $\mathbb{F}_{2^m}$ is generated, as shown in Equation 2, where $h_{ij} = \alpha_j^{i-1}/g(\alpha_j)$ for $1 \leq i \leq t, 1 \leq j \leq n$. So, to generate H, the inverse of g must be evaluated for all the n elements of the support list, $L = \{\alpha_1, ..., \alpha_n\}$. Then we form an $mt \times n$ matrix $\hat{H}$ over $\mathbb{F}_2$ by replacing each entry $u_0 + u_1 z + \cdots + u_{m-1}z^{m-1}$ of H with a column of m bits $u_0, u_1, \ldots, u_{m-1}$. This $\hat{H}$ is the PCM which must be reduced to systematic form $\hat{H} = [I_{n-k}|T]$ where I_{n-k} is the $n-k \times n-k$ identity matrix with $k = n - mt$ and T is a $n-k \times k$ matrix that is considered as the public key, PK. If this systematic form cannot be obtained, the whole key generation process needs to start over from Step 2 after overwriting δ with the new seed, δ', generated at the end of that step.

The reduction of a PCM to the systematic form is the main bottleneck in CM key generation due to two main reasons. Firstly, the size of the matrix is large, so it takes many iterations to transform the matrix into the systematic form. Secondly, not all matrices can be transformed to the systematic form, i.e. it is only possible if the right-most $k \times k$ block of that matrix consists of linearly independent columns, which is the case for about 30% of the parity check matrices for Goppa codes. Regarding the seed, δ, less than 30% of the 32-byte seeds are suitable for generating matrices in the systematic form[8].

Step 6 - ControlBits: The order of the random permutation of elements in $\mathbb{F}_{2^m}$ that is generated in Step 3, will be part of SK, which is shown as α in 2. The CM specification suggests to represent this order as a sequence of $(2m-1)*2^{m-1}$ control bits for an in-place Beneš network, which then allows to generate the ordered list of q elements of $\mathbb{F}_{2^m}$, thus, the ordered list of n elements of support list at any time [8]. Now we can make the SK, which consists of five parts:

- δ: the 32-byte string representing the seed, used as input to Step 2.

- c: the 8-byte column selection string, which is only needed in the semi-systematic form and, therefore, not important for our implementation, which focuses on the systematic form.
- g: the $t\lceil m/8 \rceil$-byte string representing the Goppa polynomial, which is generated in Step 4.
- α: the $\lceil (2m-1)2^{m-4} \rceil$-byte string representing the ordering of the generated integer list at Step 3 , which is generated in Step 6.
- s: $\lceil n/8 \rceil$-byte random string, defined in Step 2.

3 Related Work

Since the submission of CM to the NIST post-quantum cryptography competition, several investigations have been conducted to accelerate its implementation. CM key generation as the most computationally intensive part of CM is the focus of some works [33, 34, 37, 38, 23, 11, 24, 36]. Optimization of key encapsulation/decapsulation is explored in [34, 27, 31, 11, 14]. An innovative hardware architecture for the McEliece cryptosystem is introduced in [30], highlighting the advantages of modern FPGAs in addressing performance issues related to post-quantum cryptography. Wang et al. [33] present an efficient FPGA implementation of the key generation process for Niederreiter cryptosystems using binary Goppa codes. Later, they improve this implementation and also propose constant-time and fully parameterized modules for encryption and decryption in [34]. Basu et al. [7] leverage High-Level Synthesis (HLS) to create hardware designs for NIST Round 2 PQC candidates, targeting both FPGA and ASIC platforms, while investigating the impact of loop unrolling and pipelining on each algorithm's latency. A constant-time hardware implementation of CM, including key generation, encapsulation, and decapsulation, is proposed and evaluated on Xilinx Artix7 FPGA in [11]. The authors of [15] boost CM performance with an HLS-based Hardware/Software (HW/SW) co-design, implementing three FPGA accelerators for the most demanding parts, including a Gaussian systemizer for key generation, a syndrome encoder for encapsulation, and a syndrome decoder for decapsulation. Zhu et al. [37] propose an accelerator using algorithm-hardware co-design for high-throughput CM key generation in data centers and servers. Other studies [36, 38] present the FPGA implementation of some algorithm-hardware co-design schemes based on systolic arrays to speed up CM key generation while minimizing the storage overhead caused by large-size keys. Processing In Memory (PIM) is investigated to speed up the key generation and encryption algorithms of CM in [23]. An approach for outsourcing the matrix inversion algorithm in CM key generation to the connected host system is explored in [32], eliminating the need for costly high-speed RAM to store the matrix. CM implementation on ARM Cortex-M4 was explored in [10] and [27]. Roth et al. [27] addressed CM memory limitations by generating the public key on-the-fly from the extended private key. To cut memory use, they perform the encapsulation operation as the public key streams from the peer, avoiding full buffering. Meanwhile, the authors in [10] store the public key on

the flash memory of their board and propose techniques to optimize all three processes of CM. Sim et al. [31] propose an efficient software implementation for encapsulation and decapsulation of CM on 64-bit ARMv8 processors that uses vector registers for 16-byte parallel operations and achieves a speed-up of up to 1.99× compared to their baseline implementation.

Kostablaros et al. [14] propose a 22nm ASIC implementation for an efficient and constant-time accelerator for encoding and decoding of CM. In [**?**], the authors focus on the acceleration of the loading and processing of the public key from globally-shared or accelerator-private memory systems. They also propose an accelerator for encoding combined with a streaming protocol to eliminate the necessity of storing the public key in the accelerator's private memory. Their implementation on Zynq SoC is about 2.2× faster than a 64-bit vectorized CM software baseline implementation. Although pure hardware designs like ASICs and FPGAs excel in performance by offering fast processing and low power consumption, they lack flexibility for changes. HW/SW co-designs split tasks between hardware for performance and software for flexible changes, balancing speed with adaptability at the cost of performance and power consumption overhead compared to pure hardware designs. We are the first to thoroughly explore and implement RISC-V based HW/SW co-design acceleration for CM key generation. We consider scalar implementations as well as implementations with standard and custom vector extensions. To our knowledge, RVV1.0 has not yet been considered for implementation as an accelerated version of the whole CM key generation process. Only a simulation model of a RISC-V processor supporting RVV1.0 is employed in [24] to explore vectorization on GE.

4 Our Approach

In this section, we explain how vectorization can speed up `mceliece348864` key generation on RISC-V platforms. For our evaluation, we use the architecture in Figure 1, where the Vicuna coprocessor [25] is connected to an Ibex processor [17]. Ibex is a 32-bit RISC-V soft-core with two pipeline stages. It executes non-vector (scalar) instructions and forwards vector instructions to the Vicuna coprocessor. This configurable and timing-predictable 32-bit in-order vector coprocessor executes integer and fixed-point RVV instructions. Users can configure the number of parallel vector pipelines in Vicuna and the execution units in each pipeline in four modes: Compact, Dual, Triple, and Legacy. More vector pipelines can enhance the performance of computationally intensive algorithms. However, the memory bandwidth may limit the achievable throughput. In this work, we use Vicuna's compact layout without any data/instruction caches and without branch prediction. In addition, we consider a 32-bit memory bandwidth and one cycle of access latency. In the following subsections, we explain the three design and implementation approaches that we followed.

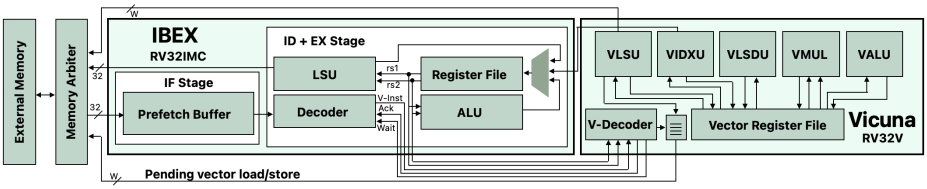

Fig. 1: Our architecture, consisting of an Ibex coupled with the Vicuna [25]

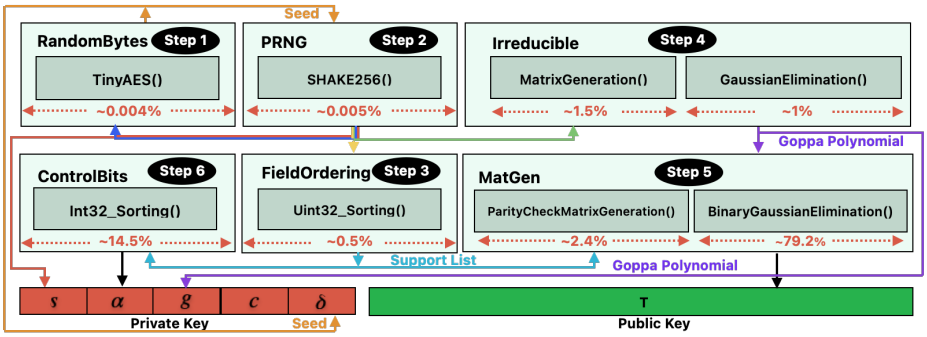

Fig. 2: Key generation profiling of the `ref` implementation of `mceliece348864`

4.1 Classic McEliece Scalar Implementation

We compile the `ref` code of `mceliece348864` for the RV32IMC architecture with the RISC-V GNU Toolchain and run it on the Ibex core to identify performance bottlenecks. Figure 2 illustrates the execution time of each function in the `ref` implementation, expressed as a percentage of the total key generation time, assuming that the keys are generated in one attempt and none of the steps fail. The figure mentions the key generation steps explained in Section 2.2. TinyAES() [13] is used for RandomBytes, and SHAKE256() is used as the PRNG algorithm. The profiling results show that the execution times of TinyAES() and SHAKE256() are negligible compared to the other functions (0.00375% and 0.0003%, respectively). Therefore, we do not consider these functions as candidates for acceleration. However, the RVV-accelerated TinyAES() proposed in [19] could have been used here to achieve a very small speed-up. All other functions of CM key generation are vectorized in our work.

As mentioned in Section 2.2, sorting the support list simplifies decoding in the decapsulation stage. In the `ref` code, a 64-bit sorting function (Uint64_Sorting()) is used for this purpose. However, in this study, both the Ibex and Vicuna architectures are designed to operate on 32 bits. So, we implement a constant-time 32-bit sorting function (Uint32_sorting in Figure 2). Finally, $(2m-1) * 2^{m-1}$ control bits for an in-place Beneš network are computed for the support list in Step 6. The Nassimi-Sahni [20] algorithm is used in the `ref` code for this purpose. This algorithm is based on a sorting algorithm that is applied on 32-

bit signed values (Int32_sorting() in Figure 2). As a cost-effective protection against errors in computing the control bits for permutation, it is recommended that implementers verify, after computation, that applying the Beneš network results in their desired permutation; otherwise, restart the key generation [8].

As presented in Section 2.2, the generation of the Goppa polynomial in Step 4 of the key generation algorithm begins with the formation of the matrix M, where each row is a power of the element $f \in \mathbb{F}_{2^{mt}}$. This is performed in MatrixGeneration(), as indicated in Figure 2. In order to compute the powers of $f = f_{t-1}y^{t-1} + \cdots + f_1 y + f_0$, with $f_i \in \mathbb{F}_{2^m}$ for $0 \leq i \leq t-1$, we perform repeated multiplications. The multiplication of two elements in $\mathbb{F}_{2^{mt}}$ (`GF_mul`), can be considered as multiplying two polynomials of degree $t-1$ with coefficients in $\mathbb{F}_{2^m}$ followed by reduction by $F(y)$. This can be represented by the multiplication of two matrices as in Figure 3 and then reduction of P_i values for $0 \leq i \leq 2t-2$ by $F(y)$ that is $F(y) = y^{64} + y^3 + y + x$ for `mceliece348864`.. Here $f \in \mathbb{F}_{2^{mt}}$ is multiplied with itself and $\otimes$ represents multiplication in $\mathbb{F}_{2^m}$(`gf_mul`). Hence, squaring an element $f \in \mathbb{F}_{2^{mt}}$ (without reduction) needs $t \times t = t^2$ `gf_mul` and forming matrix M needs $(t-1) \times t^2$ `gf_mul`.

$$\begin{pmatrix} P_0 \\ P_1 \\ P_2 \\ P_3 \\ . \\ \vdots \\ P_{2t-3} \\ P_{2t-2} \end{pmatrix} = \begin{pmatrix} 0 & 0 & \dots & 0 & 0 & f_0 \\ 0 & 0 & \dots & 0 & f_0 & f_1 \\ 0 & 0 & \dots & f_0 & f_1 & f_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ f_0 & f_1 & \dots & f_{t-3} & f_{t-2} & f_{t-1} \\ f_1 & f_2 & \dots & f_{t-1} & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ f_{t-1} & 0 & \dots & 0 & 0 & 0 \end{pmatrix} \otimes \begin{pmatrix} f_{t-1} \\ f_{t-2} \\ f_{t-3} \\ \vdots \\ f_2 \\ f_1 \\ f_0 \end{pmatrix}$$

Fig. 3: Multiplication of two matrices with elements in $\mathbb{F}_{2^m}$

The ParityCheckMatrixGeneration() in Figure 2 generates matrix H elements in equation 2 where $h_{ij} = \alpha_j^{i-1}/g(\alpha_j)$ for $1 \leq i \leq t, 1 \leq j \leq n$. If we represent the Goppa polynomial, g, as $g(x) = g_t x^t + g_{t-1}x^{t-1} + ... + g_1 x + g_0$, where $g_i \in \mathbb{F}_{2^m}$ for $0 \leq i \leq t$ and $g_t = 1$, then evaluating it at $\alpha_j \in \mathbb{F}_{2^m}$, for $1 \leq j \leq n$ requires computation: $g(\alpha_j) = {\alpha_j}^t \oplus (g_{t-1} \otimes {\alpha_j}^{t-1}) \oplus ... \oplus (g_1 \otimes \alpha_j) \oplus g_0$, $g(\alpha_j) \in \mathbb{F}_{2^m}$, where $\oplus$ and $\otimes$ represents addition and multiplication in $\mathbb{F}_{2^m}$, respectively. This needs $\mathcal{O}(t^2)$ multiplication and t addition; however, by using Horner's rule, it needs only t multiplication with the same number of addition:

$g(\alpha_j) = ((((((g_{t-1} \oplus \alpha_j) \otimes \alpha_j) \oplus g_{t-2}) \otimes \alpha_j) \oplus ... \oplus g_1) \otimes \alpha_j) \oplus g_0$

Based on Fermat's little theorem [18], we can compute A^{2m-2} as the modular inverse of any non-zero element $A \in \mathbb{F}_{2^m}$. In `mceliece348864` with $m = 12$ it is $A^{-1} = A^{4094}$ where $A \in \mathbb{F}_{2^{12}}$ and $A \neq 0$. Computing A^{4094} with the square-and-multiply approach needs 11 square operations and 10 multiplications; however, with the help of the addition chain method [9], we need the following powers of A, which results in 11 square operations and only five multiplications: 1→2→3→6→12→15→30→60→120→240→255→510→1020→1023→2046→2047→4094.

Each square operation and each multiplication is implemented 25 and 60 RISC-V integer instructions, respectively. Thus, the A^{-1} computation where $A \in \mathbb{F}_{2^{12}}$ needs around 575 RISC-V integer instructions in total.
After computation of $g^{-1}(\alpha_j)$ for $1 \leq j \leq n$, to generate each row of the PCM, we need to multiply them with corresponding α_j^i for $1 \leq i \leq t-1$ which needs $n \times (t-1)$ `gf_mul`. Totally, generating matrix H needs $(5 \times n) + (t \times n) + ((t-1) \times N) = 2 \times N \times (t+2)$ `gf_mul` that equals 460416 times in the case of `mceliece348864` . Later, the PCM must be reduced to systematic form. One way to do that, which we adopt in our work, is to use Gaussian Elimination to put that matrix in reduced row-echelon form (RREF) and return the result if it is in the systematic form [8]. RREF matrices have three main properties: each non-zero row begins with a leading 1, which is called a Pivot element; the leading 1 of each row is at the right of the leading 1 of the row above (Staircase Form), and all-zero rows are at the bottom of the matrix. To transform a matrix into RREF, one needs to identify the pivot element in each column, form it into one, and then bring all entries below and above the pivot element to zero. The BinaryGaussianElimination() in Figure 2 uses the constant-time Algorithm 1 for this purpose in the `ref` C code. This algorithm operates on a binary matrix, which is generated in Step 5 of the key generation algorithm. The scalar implementation of this algorithm takes about 12.8 seconds at Freq=100MHz.

Algorithm 1 Binary Gaussian Elimination in Classic McEliece [8]

Input: Binary matrix $\hat{H}$ of size $mt \times n$
Output: Binary matrix $\hat{H}$ in form $(I_{n-k}|T)$ or $\perp$

for $i = 0$ to $\frac{(mt+7)}{8} - 1$ **do**
  for $j = 0$ to 7 **do**
    $row \leftarrow i \times 8 + j$
    if $row \geq mt$ **then**
      break
    for $k = row + 1$ to $mt - 1$ **do** ▷ loop-1
      $mask \leftarrow -(((\hat{H}[row][i] \oplus \hat{H}[k][i]) \gg j)\&1)$
      for $c = 0$ to $n/8 - 1$ **do** ▷ loop-1-1
        $\hat{H}[row][c] \leftarrow (\hat{H}[row][c]) \oplus (\hat{H}[k][c]\&mask)$
    if $((\hat{H}[row][i] \gg j)\&1) == 0$ **then**
      return -1 ▷ Not systematic
    for $k = 0$ to $mt - 1$ **do** ▷ loop-2
      if $k \neq row$ **then**
        $mask \leftarrow -((\hat{H}[k][i] \gg j)\&1)$
        for $c = 0$ to $n/8 - 1$ **do** ▷ loop-2-1
          $\hat{H}[k][c] \leftarrow \hat{H}[k][c] \oplus (\hat{H}[row][c]\&mask)$

4.2 Standard Vector Accelerated (`sva`) Classic McEliece

Compiling the `ref` C code for RV32IMCV, with the -O3 optimization flag, generates a vectorized binary file automatically (auto-vectorized). In some cases, the auto-vectorized binary file is not executable due to some RVV instructions not being supported by Vicuna; hence, we use RISC-V vector V intrinsics [1] to

manually vectorize the `ref` C code (manually-vectorized). The RISC-V vector C intrinsics offer a C-language interface for directly utilizing the RVV instructions, with assistance from the compiler in handling instruction scheduling and register allocation. Additionally, these intrinsics simplify development by handling vector configuration settings, reducing the burden on users. As we explained in Section 4.1, matrix M generation at Step 4 and PCM generation at Step 5, need $(t-1) \times t^2$ and $2 \times N \times (t+2)$ `gf_mul`, respectively which are equal to 258048 and 460416 in the case of `mceliece348864`. By vectorization with the vector configuration VLEN=256, ELEN=16, and LMUL=4, we can compute `gf_mul` of 64 elements (vl=64) in parallel. This results in 1.77× and 1.6× speed-up in MatrixGeneration() and ParityCheckMatrixGeneration() functions in Figure 2, respectively. Important candidates for vectorization are loops. Binary Gaussian Elimination (BGE) in Algorithm 1 contains two main loops, $loop-1$ and $loop-2$, where firstly, some masks are computed, and then, based on those masks, the matrix entries are changed. The loop-1 iterates on all matrix rows below the target, and loop-1-1 iterates on all columns in the row selected by loop-1. In loop-1-1 and loop-2-1, the values of row and $mask$ are fixed. When VLEN=256 bits and ELEN=8 bits, the number of elements in a vector register group are 32, 64, 128, and 256 bytes by adopting LMUL=1 ($m1$), LMUL=2 ($m2$), LMUL=4 ($m4$), and LMUL=8 ($m8$), respectively. By loading multiple bytes of a matrix row into a vector register, the computation can be applied to several columns simultaneously. The auto-vectorized BGE with VLEN=256 bits, running on our platform at a frequency of 100MHz, takes about 8.43 seconds. In this auto-vectorized version, the compiler processes each of $loop-1$ and $loop-2$ in 14 iterations with the vector configuration LMUL=1. In Manually-vectorized mode, we evaluate the effect of changing the vector length multiplier parameter, LMUL_i, at each iteration $1 \leq i \leq I$, on the execution time of BGE in `mceliece348864`. We compute multiple masks using RVV results, showing that this would yield a higher cycle count than computing masks sequentially in scalar mode. Thus, we only focused on accelerating loop-1-1 and loop-2-1 by means of vector instructions. The results are shown in Table 1. The highest speed-up is 2.46x for using three iterations with $\text{LMUL}_1 = 8$, $\text{LMUL}_2 = 4$, and $\text{LMUL}_3 = 2$.

4.3 Custom Vector Accelerated (`cva`) Classic McEliece

By defining a custom instruction, a complete operation that needs multiple integer or standard RVV instructions can be executed just by calling one instruction at the expense of a small hardware overhead. Three custom vector instructions are proposed to accelerate `mceliece348864` key pair generation beyond the vectorized design using standard RVV. These includes: `vgfmul` for vector multiplication in $\mathbb{F}_{2^{12}}$, `vgfsq` for vector square operation in $\mathbb{F}_{2^{12}}$, `vgfinv` for vector modular inversion over $\mathbb{F}_{2^{12}}$. The software and hardware must be changed to use these custom instructions. Among the four RISC-V instruction formats, R, I, S, and U, we use the R-type instruction for the custom extension, with format $func7|vs2|vs1|func3|vd|opcode$. The $opcode$ is set to $0X0B$ for all these custom vector instructions. Values in $func7$ and $func3$ fields differentiate between those

Table 1: Speed-up and code size overhead of Binary Gaussian Elimination in `sva` (Auto-vectorized/Manually-vectorized) and `ref` at a frequency of 100MHz.

Iteration Count(I)	$LMUL_i$ for $i \in [1...I]$ m8	m4	m2	m1	Cycles (Mcyc.)	time (sec)	speed-up (x)	code size (Byte)	overhead (%)
Scalar implementation (ref)									
-	-	-	-	-	1280	12.798	1	602	0
Auto-vectorized implementation (based on sva)									
14	-	-	-	$i \in [1...14]$	843	8.430	1.52	1692	181.06
Manually-vectorized implementation (based on sva)									
14	-	-	-	$i \in [1...14]$	763	7.628	1.68	562	-6.64
7	-	-	$i \in [1...7]$	-	559	5.587	2.29	610	1.33
8	-	-	$i \in [1...6]$	$i = 7, 8$	592	5.926	2.16	714	18.60
4	-	$i = 1, 2, 3, 4$	-	-	597	5.973	2.14	670	11.30
4	-	$i = 1, 2, 3$	$i = 4$		528	5.278	2.42	610	1.33
4 (L-unrolled)	-	$i = 1, 2, 3$	$i = 4$		538	5.384	2.38	4116	583.72
5	-	$i = 1, 2, 3$	-	$i = 4, 5$	562	5.617	2.28	714	18.60
2	$i = 1, 2$		-	-	581	5.814	2.20	562	-6.64
3	$i = 1$	$i = 2, 3$	-	-	595	5.947	2.15	676	12.29
3	$i = 1$	$i = 2$	$i = 3$		520	5.198	**2.46**	676	12.29
4	$i = 1$	$i = 2$		$i = 3, 4$	554	5.537	2.31	676	12.29

custom instructions. The values of $vs1$, $vs2$, and vd define the input and output vector registers. There are two ways to add custom instructions to software. In the first approach, the compiler, GCC in our work, is modified based on the new instructions, so any custom instruction would be automatically translated to its binary form during compilation. The second approach is using the *.insn* directive with inline assembly with a format *.insn R opcode*, $func3, func7, vd, vs1, vs2$. This directive allows the numeric representation of an instruction and directs the assembler to insert the operands accordingly. Changing the Compiler is a complex task as it needs code modifications and may lead to compatibility issues. As addressing these intricacies requires significant effort and testing, the first approach is beyond the scope of our work. The second approach is challenging to manage since we use RISC-V Vector Intrinsics in this work. These RVV Intrinsics rely on the compiler to determine the vector registers used as arguments for each instruction; their exact allocation is not explicitly known and cannot be directly specified with the *.insn* directive. One solution is to use the intrinsic of any instruction with the same format of the custom instructions (R-type) and, after compilation, change the required fields such as *opcode*, $func7$, and $func3$ in binary output. At the RTL level, we need to add logic to the decoding stages in both Ibex and Vicuna. Hence, the Ibex decoder recognizes the opcode of the custom vector instructions and forwards them to Vicuna. The Vicuna decoder decides to activate the proper execution unit based on the values of $func7$ and $func3$. All the proposed custom instructions are implemented by combinational circuits except `vgfinv`.

Modular multiplication in $\mathbb{F}_{2^m}$ can be carried out in different ways. The basic method is the "shift-and-add," which processes the bits of one operand from left to right and adds shifted versions of another operand together to compute the result. This method needs $m-1$ shift operations and m additions. Here, we use a more optimal multiplication method, namely the Lopez-Dahab Multiplier[16],

Algorithm 2 Lopez-Dahab Multiplication [16] in $\mathbb{F}_{2^{12}}$

Input: $A = [a_{11}, \ldots, a_0]$, $B = [b_{11}, \ldots, b_0]$, $f(x)$: Field Irreducible Polynomial
Output: $C = [c_{11}, c_{10}, \ldots, c_0] = A \cdot B \pmod{f(x)}$

1: Make a look-up table, $T[u]$ for $0 \leq u \leq 7$, using the equation $T[u](x) = (u_2x^2 + u_1x + u_0)A(x)$ where $u = (u_2u_1u_0)_2$
2: $D[23:0] \leftarrow 0$
3: **for** $n = 0$ to 9 by 3 **do**
4: $\quad k \leftarrow (b_{n+2}b_{n+1}b_n)_2$
5: $\quad D \leftarrow D \oplus \{T[k] \ll n\}$
6: $C \leftarrow D \pmod{f(x)}$

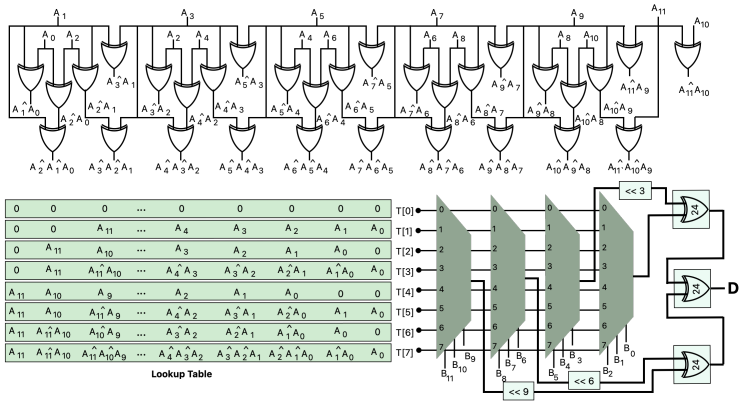

Fig. 4: Multiplication over the field $\mathbb{F}_{2^{12}}$ in hardware

as shown in Algorithm 2. The basic idea of this method is to compute multiples of one operand for certain offsets and save them into a look-up table. We chose 3-bit (0 to 7) for the range of offset, so the look-up table has the size of $8 \times (m+2)$, where m is the size of operands. Then, this table is accessed by addresses that are formed of consecutive 3 bits of the second operand; the values obtained from the table are shifted accordingly and XORed with the intermediate result.

For modular reduction, given that the irreducible polynomial is fixed for each CM parameter set, we can design hardware with simple bit-wise operations. For `mceliece348864` this becomes:
$C = [c_{11}, c_{10}, \ldots, c_0] = D \pmod{f(x)} = [d_{11} \oplus d_{20}, d_{10} \oplus d_{22} \oplus d_{19}, d_9 \oplus d_{21} \oplus d_{18}, d_8 \oplus d_{20} \oplus d_{17}, d_7 \oplus d_{19} \oplus d_{16}, d_6 \oplus d_{18} \oplus d_{15}, d_5 \oplus d_{17} \oplus d_{14}, d_4 \oplus d_{16} \oplus d_{13} \oplus d_{22}, d_3 \oplus d_{15} \oplus d_{12} \oplus d_{21}, d_2 \oplus d_{14}, d_1 \oplus d_{13} \oplus d_{22}, d_0 \oplus d_{12} \oplus d_{21}]$

By inserting a zero bit between consecutive bits of $A \in \mathbb{F}_{2^{12}}$ and applying modular reduction we can compute $A^2 \pmod{f(x)}$. By repeating this approach, we can also generate A^4, A^{16}. Figure 5 shows how inversion in $\mathbb{F}_{2^{12}}$ is implemented in hardware. This pipelined implementation has a latency of 4 clock cycles.

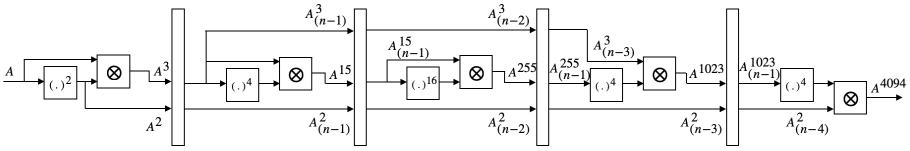

Fig. 5: Inversion over the field $\mathbb{F}_{2^{12}}$ in hardware

5 Results

In this study, the RISC-V GNU Compiler Toolchain[4] with the -O3 optimization flag is used to compile our software. We use Vivado 2021.2 to synthesize and implement the design at a frequency of 100MHz for the Xilinx Alveo U250 Data Center accelerator card. First, the `ref` code for `mceliece348864` is compiled for RV32IMC and profiled for execution on the Ibex core. Regarding these results, the best candidates for acceleration are MatGen, which contains the BinaryGaussianElimination(), ControlBits, and Irreducible algorithms, while the TinyAES() and SHAKE256() execution times are negligible in comparison with the other functions. Then, we propose vectorized variants for these functions using standard instructions in RVV1.0 and, where possible, custom vector instructions. The results for the `sva` implementation and the vector-accelerated variants (`sva` and `cva`) are shown in Table 2. To the best of our knowledge, the only other work that uses RVV for accelerating CM key generation is [24], where the Binary Gaussian Elimination is compiled for RV64IV, with VLEN = 1024, and an evaluation of the speed-up for various memory port widths (W) from 64 to 256 bits and different cache configurations. With a cache miss penalty of 10 cycles per 64-bit memory access, they achieve a 6 × speed-up for GE, while with a 32-bit memory interface without any cache and with VLEN=256, we reached a speed-up of 2.46 ×. Two hardware implementations for McEliece and Niederreiter key generation are represented in [30] and [34], respectively, for parameter sets that are different from ours. Hence, we could not directly compare the results. The works [11, 38] implemented the key generation of `mceliece348864` in hardware. Table 3 shows their results. A pure hardware implementation of CM key generation definitely surpasses a hardware/software co-design. However, our approach is more flexible. H t a h

6 Conclusion

This paper presents a hardware/software co-design acceleration of Classic McEliece key generation based on vectorization. We profile all the functions in a scalar implementation of the NIST `ref` implementation, published by the authors of CM, and we optimize the most computationally intensive functions using standard and custom vector instructions. We explore various vector configurations

[4] https://github.com/riscv-collab/riscv-gnu-toolchain

Table 2: Implementation results for the scalar (`ref`), the standard vector-accelerated (`sva`) and the custom vector-accelerated (`cva`) implementations of `mceliece348864` key generation at a frequency of 100MHz

Function	Cycle count(Mcyc.)			Execution time(ms)(speed-up)			Code Size (B)		
	`ref`	`sva`	`cva`	`ref`	`sva`	`cva`	`ref`	`sva`	`cva`
TinyAES()	0.0065	-	-	0.065	-	-	2260	-	-
SHAKE256()	0.084	-	-	0.804	-	-	7434	-	-
FieldOrdering	6.5	5.8	-	65.89	58.288 (1.13x)	-	598	1442	-
Irreducible	36.4	21.8	2.7	363.9	218.323 (1.67x)	26.648 (13.66x)	3682	5068	604
MatrixGen.()	21.6	12.2	2.7	215.93	121.43 (1.78x)	10.6 (20.38x)	1200	1940	368
GE()	14.8	10.2	-	148	101.7(1.45x)	-	2306	3142	376
MatGen	1337.8	558.9	527.4	13378.28	5589.991 (2.39x)	5273.766 (2.54x)	3310	7620	2816
PCMGen.()	38.7	24.6	7.3	387	246(1.57x)	73(5.3x)	2544	6986	2292
BGE()	1279.7	519.8	-	12797.53	519.452 (2.46x)	-	602	656	-
ControlBits	782.4	397.6	-	7824.2	3976.35 (1.97x)	-	1918	5576	-

Table 3: Comparison of existing `mceliece348864` key generation

work	platform (FPGA)	Freq (MHz)	Cycles (Kcyc.)	Time (ms)	Utilization LUT	FF	BRAM	Architecture
[38]	Xilinx Artix-7 [a,e]	151	255	1.69	58208	63561	77	Pure HW
[38]	Xilinx UltraScale+ [a,e]	278	183	0.66	69428	69636	64.5	Pure HW
[11]	Xilinx Artix-7 [b]	142	1000	6.8	25822	37185	165	Pure HW
[11]	Xilinx UltraScale+ [b,c]	186	1000	5.2	25463	37245	112.5	Pure HW
This Work (`cva`)	Xilinx UltraScale+ [a,d]	100	$536e^3$	$5.36e^3$	9486	6769	64	Ibex+Vicuna
This Work (`ref`)	Xilinx UltraScale+ [a,d]	100	$1381e^3$	$13.8e^3$	3226	1858	0	Ibex

a: shake is included in utilization; b: shake is not included in utilization;
c: Xilinx Zynq UltraScale+; d: Xilinx Alveo U250;
e: systolic array architecture for GF(2) systemizer;

based on an Ibex processor in combination with the Vicuna vector coprocessor. Our proposed vectorized design using standard vector instructions shows a speed-up of about 2×. We achieve a speed-up between 2.5× and 13× for different functions in the design based on custom vector instructions. To the best of our knowledge, this is the first work that explores standard and custom vectorization of the Classic McEliece key generation algorithm and evaluates the implementation results on an FPGA.

References

1. RISC-V Vector C Intrinsic, https://github.com/riscv-non-isa/rvv-intrinsic-doc
2. Spike RISC-V ISA Simulator, https://github.com/riscv-software-src/riscv-isa-sim
3. NIST, FIPS 202: SHA-3 Standard (2015). https://doi.org/10.6028/NIST.FIPS.202
4. RISC-V Vector Extension,version 1.0 (2021), https://github.com/riscvarchive/riscv-v-spec/releases/download/v1.0/riscv-v-spec-1.0.pdf
5. Classic McEliece: Conservative code-based cryptography: design rationale (2022), https://classic.mceliece.org/mceliece-rationale-20221023.pdf
6. NIST, FIPS 186-5: Digital Signature Standard (DSS) (2023), https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf
7. Basu, K., et al.: NIST Post-Quantum Crypto Hardware Study (2019)

8. Bernstein, D., et al.: Classic McEliece Crypto. Tech. rep., NIST (2022), https://classic.mceliece.org/mceliece-spec-20221023.pdf
9. Brauer, A.: On addition chains (1939)
10. Chen, M.S., Chou, T.: Classic McEliece on the ARM cortex-M4. TCHES (2021)
11. Chen, P.J., et al.: Complete and improved FPGA implementation of Classic McEliece. TCHES (2022)
12. Dekkaki, K.C., et al.: Exploring post-quantum cryptography: Review and directions for the transition process. Technologies **12**, 241 (2024)
13. Kokke: tiny-AES.c (2021), https://github.com/kokke/tiny-AES-c
14. Kostalabros, V., et al.: Safety-Critical RISC-V Classic McEliece Accelerator. In: ARC. pp. 282–295 (2024)
15. Kostalabros, V., et al.: HLS-Based HW/SW Co-Design of the Post-Quantum Classic McEliece Cryptosystem. In: FPL. pp. 52–59. IEEE (2021)
16. López, J., et al.: High-Speed Software Multiplication in F2m. In: indocrypt (2000)
17. lowRISC: Ibex: An embedded 32 bit RISC-V CPU core (2018)
18. Menezes, A.J., et al.: Handbook of Applied Cryptography. CRC Press (1996)
19. Namazi Rizi, M., et al.: Optimised AES with RISC-V Vectors. In: DDECS (2024)
20. Nassimi, D., Sahni, S.: Benes network setup. IEEE Trans. Comput. **C–31** (1982)
21. NIST: Announcing the advanced encryption standard (AES) (2001)
22. NIST: PQC Standardization (2017), https://csrc.nist.gov/pqc-standardization
23. Nugier, C., Migliore, V.: Acceleration of Classic McEliece Post-Quantum Cryptosystem with Cache Processing. IEEE Micro (2023)
24. Pircher, S., et al.: Exploring the RISC-V vector extension for the classic mceliece post-quantum cryptosystem. In: ISQED. pp. 401–407 (2021)
25. Platzer, M., Puschner, P.: Vicuna: A Timing-Predictable RISC-V Vector Coprocessor for Scalable Parallel Computation. In: ECRTS (2021)
26. Rivest, R.L., et al.: A method for obtaining digital signatures and public-key cryptosystems. Communications of the ACM **21**(2), 120–126 (1978)
27. Roth, J., Karatsiolis, E., Krämer, J.: Classic mceliece implementation with low memory footprint. In: CARDIS. pp. 34–49. Springer (2021)
28. Schwabe, P., et al.: Post-Quantum TLS without Handshake Signatures. In: CCS. pp. 1461–1480 (2020)
29. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J. Comput. (1997)
30. Shoufan, A., et al.: A Novel Cryptoprocessor Architecture for the McEliece Public-Key Cryptosystem. IEEE Trans. Computers **59**, 1533–1546 (2010)
31. Sim, M., et al.: Efficient Implementation of the Classic McEliece on ARMv8 Processors. In: WISA. pp. 324–337. Springer (2023)
32. Urian, R., Schermann, R.: Classic McEliece Key Generation on RAM constrained devices. IACR Cryptol. ePrint Arch. (2022)
33. Wang, W., et al.: FPGA-based key generator for the niederreiter cryptosystem using binary goppa codes. In: CHES. pp. 253–274. Springer (2017)
34. Wang, W., et al.: FPGA-based Niederreiter cryptosystem using binary Goppa codes. In: PQCrypto. pp. 77–98. Springer (2018)
35. Waterman, A., et al.: Volume I: Unprivileged ISA p. 238 (2019)
36. Zhang, H., et al.: Fast Hardware for McEliece Key Generation. IEEE Trans. Circuits Syst. I **72**, 1321–1331 (2025)
37. Zhu, Y., et al.: Compact GF(2) systemizer and optimized constant-time hardware sorters for Key Generation in Classic McEliece. Crypt. ePrint Arch. (2022)
38. Zhu, Y., et al.: Mckeycutter: A High-throughput Key Generator of Classic McEliece on Hardware. In: DAC (2023)